DEBRE BERHAN UNIVERSTITY

College of Computing

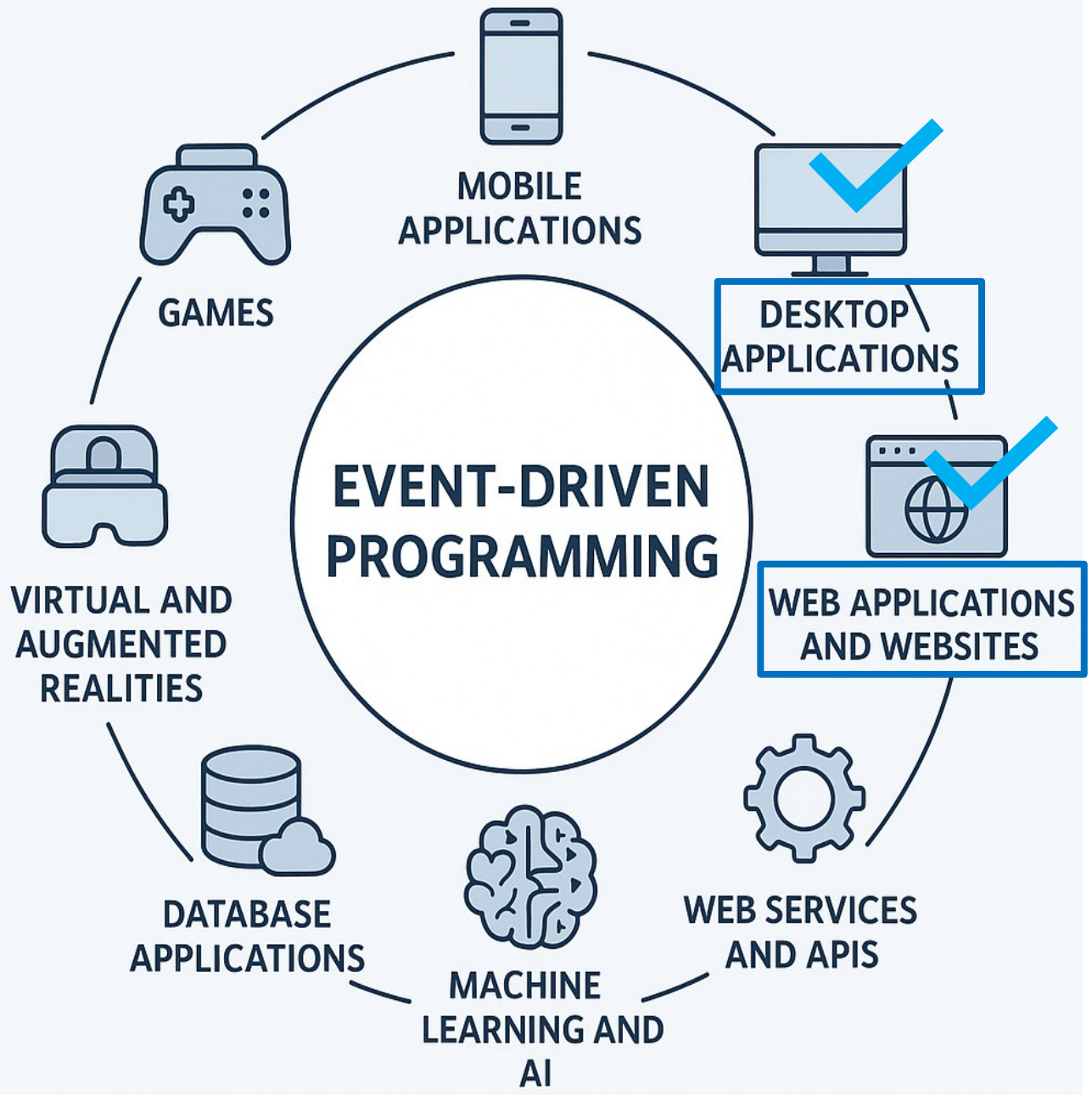
Chapter 1: Introduction (Event-Driven Fundamentals)

By Worku Muluye

August 13, 2026

Outline

- Software Development
- Event Driven Programming Paradigm
- Introduction to .NET Framework
  - .NET Framework Architecture
  - .NET Framework Components



Software Development

- Software development is the process of:
  - Designing,
  - Creating,
  - Testing, and
  - Maintaining applications, frameworks, or other software components.
- It transforms user requirements into functional software solutions that solve problems or improve efficiency.
- The goal is to build a high-quality, reliable product that meets user requirements.

Software Development Life Cycle

- **Planning:** Define project goals, scope, and objectives.
  - Conduct feasibility studies and gather initial requirements to estimate costs, timelines, and resources.
- **Requirements Gathering and Analysis:** Collect and document detailed specifications, resulting in a Software Requirements Specification (SRS) that guides development.
- **Design:** Create the system architecture, UI, and data models to meet the defined requirements.

Software Development Life Cycle

- **Implementation and Coding(Development):** Writing the actual code using a programming language and a framework.
  - Developers code the software based on design documents, adhering to coding standards and using collaborative tools.
- **Testing:** QA engineers perform unit, integration, system, and acceptance testing to ensure functionality, security, correctness, and quality.
- **Deployment:** Releasing the software to users.
- **Maintenance:** Updating, fixing, and improving the software over time.

Software Development Approaches

- **Waterfall Model:** Sequential, phase-based development
  - Each stage must finish before the next begins.
  - Best for projects with well-defined requirements.
- **Agile:** An iterative and adaptive approach, and it emphasizes flexibility, collaboration, and customer feedback.
  - It is suitable for projects with changing requirements.
- **DevOps:** Integrates development and operations.
  - It focuses on automation, continuous integration, and delivery (CI/CD).
  - It improves collaboration and enables faster releases.

Event Driven Programming Paradigm

- **Programming Paradigm:** A fundamental style, or way, of thinking about and structuring a computer program.
- It provides a conceptual framework that defines how a developer approaches a problem and organizes the solution.
- A paradigm is not a specific language but rather a set of principles and patterns that a language may be built to support.
- Many modern programming languages are multi-paradigm.

Event Driven Programming Paradigm

- **Common Programming Paradigms are: technique to write a program**
  - **Procedural Programming:** Focused on procedures or functions that operate on data, e.g., C.
  - **Object-Oriented Programming:** Focused on objects that contain both data and methods, for instance, Python, Java, C#, C++, ...
  - **Functional Programming:** Treat computation as the evaluation of mathematical functions, for instance, Haskell, F#, ...
  - **Event-Driven Programming**: The flow of the program is determined by events, e.g., C#.

Event Driven Programming Paradigm

- **.NET Programming Languages:**
- C#: an object-oriented language that has a syntax that's similar to C++, Java, and JavaScript.
- Visual Basic: An object-oriented language that has a syntax that's designed to be easy to learn and use.
- F#(F-sharp): A language that combines functional, procedural, and object-oriented programming

Event-Driven Programming(EDP)

- EDP is a programming paradigm in which the flow of the program is determined by events, such as:
  - User actions (clicks, key presses, mouse movements),
  - Sensor outputs, or
  - Messages from other programs or threads.
- Widely used in graphical user interfaces, real-time systems, and modern application frameworks such as .NET, JavaScript, and JavaFX.

Event-Driven Programming(EDP)

- **Events:** An event is an action or occurrence that the program can recognize and respond to. Events can be generated by:
  - **User actions:** A button click, a mouse movement, or a keystroke.
  - **System events:** A file completing its download, a network request returning data, or a timer expiring.
  - **Custom events:** An application-specific trigger, such as a game character leveling up or a stock price crossing a certain threshold.
- **Event sources (Producers):** generate and publish events.
  - In a web browser, a button element is an event source that emits a **click** event.

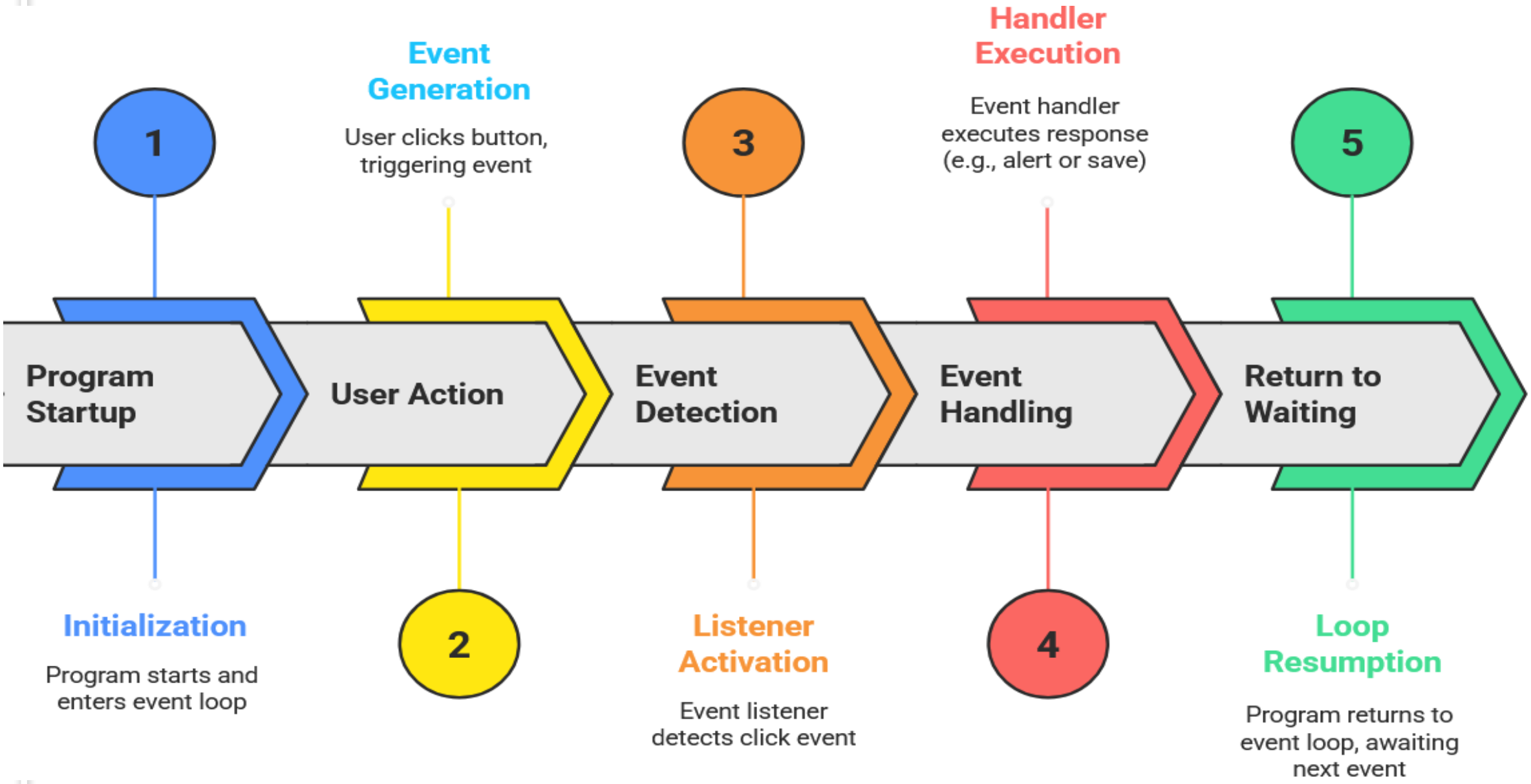
Event-Driven Programming(EDP)

- **Event listeners (Consumers):** procedures or functions that are configured to listen for specific events.
  - When a listener detects an event, it triggers an event handler.
- **Event handlers (Callbacks):** functions that contain the logic to be executed when a particular event occurs. For instance, an event handler for a button click might save data or update the user interface.
- **Event loop:** A core mechanism in many event-driven systems that constantly checks for new events and dispatches them to their corresponding handlers.
  - This allows for asynchronous, non-blocking execution.

How Event-Driven Programming Works?

- **Imagine a graphical user interface with a button:**
- **Waiting for an event:** The program starts up and enters an event loop, which is a state where it simply waits and listens for user actions.
- **Generating an event:** The user moves their mouse and clicks the button. This action is the event.
- **Detecting the event:** An event listener that was specifically attached to the button detects that a click event has occurred.
- **Handling the event:** The event listener then calls its corresponding event handler, which contains instructions on how to respond.
- **Returning to waiting:** After the event handler is finished, the program returns to the event loop, waiting for the next event.

How Event-Driven Programming Works?



What is .NET?

- **.NET** stands for Network Enabled Technology.
- **Dot (.)** represents object-oriented; **NET** represents internet.
- **.NET** means object-oriented technology for internet-based applications.
- A **Framework** is a software environment composed of integrated technologies.
- **Framework** provides tools, libraries, and runtime components for building applications that can run anywhere.
- .NET simplifies application development by providing a unified environment, language interoperability, and a rich class library.

What is .NET?

- .NET is free, open source and cross-platform development platform.
  - Free == FREE
  - Open source == We develop in the open.
  - Cross-platform == Works and runs on Windows, macOS and Linux.
- .NET supports C#, F#, Java, Python, C++ and more programming languages.
- **Tools**: visual studio and visual studio code. Absolut beginners are recommended to use visual studio code.
- What can we build on with .NET?
  - We can build almost anything such as mobile apps, desktop apps, web apps, microservices, cloud and more.

.NET Implementations

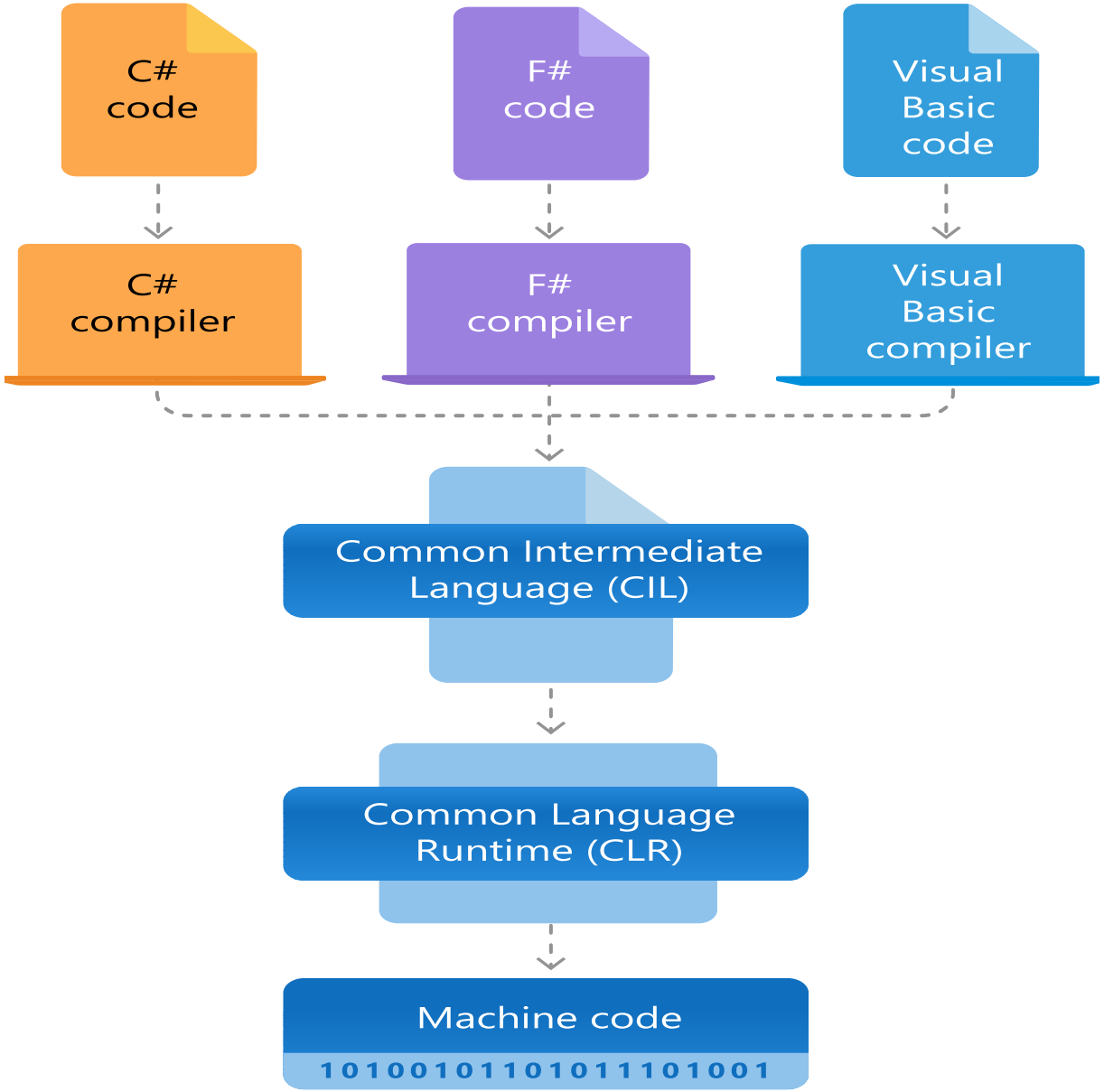
- **.NET Framework:**
  - Original implementation.
  - Runs on Windows (desktop, web, services).
- **.NET (.NET Core):**
  - Cross-platform and open-source.
  - Supports Windows, Linux, and macOS.
- **Xamarin/Mono:**
  - Designed for mobile applications on iOS and Android.

.NET Framework

- The .NET Framework is a development platform designed to build and run applications across:
  - Desktop,
  - Web,
  - Mobile,
  - XML Web services,
  - Enterprise environments.
- It is a software development platform developed by Microsoft, primarily used for building and running applications for **Windows**.

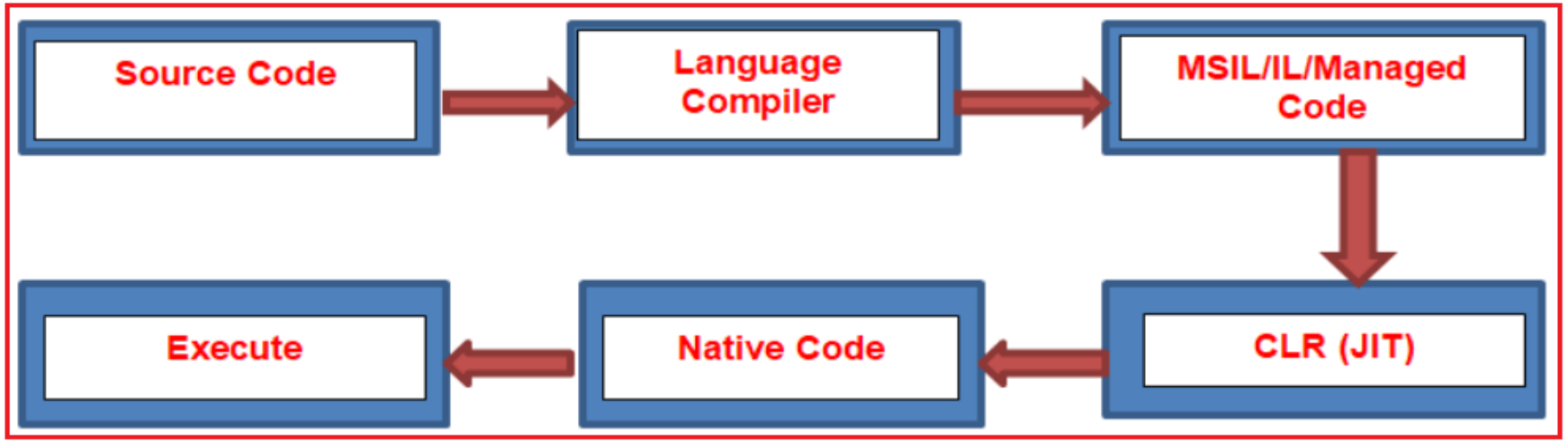
.NET Framework Architecture

- The .NET Framework Architecture is a software design model developed by Microsoft.
- It provides a managed execution environment, libraries, and language interoperability for building and running applications on Windows.
- The two major components of .NET Framework are the Common Language Runtime and the .NET Framework Class Library.



.NET Framework Components

- **Common Language Runtime (CLR):**
  - It is a program execution **engine of .NET** that loads & executes the program.
  - Converts(Microsoft Intermediate Language)MSIL (Intermediate Code) into native code.
  - It acts as an interface between the framework and the operating system.
  - It handles exceptions, memory management, and garbage collection.
  - It provides security, type-safety, interoperability, and portability.



.NET Framework Components

- **Framework Class Library(FCL):**
  - Building blocks for application development.
  - It is a standard library that contains a collection of thousands of classes, interfaces, used to build applications.
  - The BCL (Base Class Library) is the core of the FCL and provides basic functionalities for strings, dates, numbers, collections, I/O, etc.
  - It provides APIs for I/O operations, collections, data access, networking, cryptography, and XML processing.
  - It reduces development time and ensures consistent application design.

.NET Framework Components

- **Application Domains:**
  - Provide an isolation boundary within the CLR for running applications.
  - Enable multiple applications to run within a single process while maintaining independence, security, and fault isolation.
  - Allow safe unloading of applications without affecting others in the same process.

Core Application Models

- **Specialized frameworks for different application types.**
  - **Windows Forms:** a smart client technology for the .NET Framework, a set of managed libraries that simplify common application tasks such as reading and writing to the file system.
  - **ASP.NET:** a web framework designed and developed by Microsoft, used to create websites, web applications, and web services.
  - **ADO.NET:** a module of the .NET Framework, used to establish a connection between an application and data sources such as MS SQL Server and XML.
    - It consists of classes that can be used to connect, retrieve, insert, and delete data.

Extended Application Modules

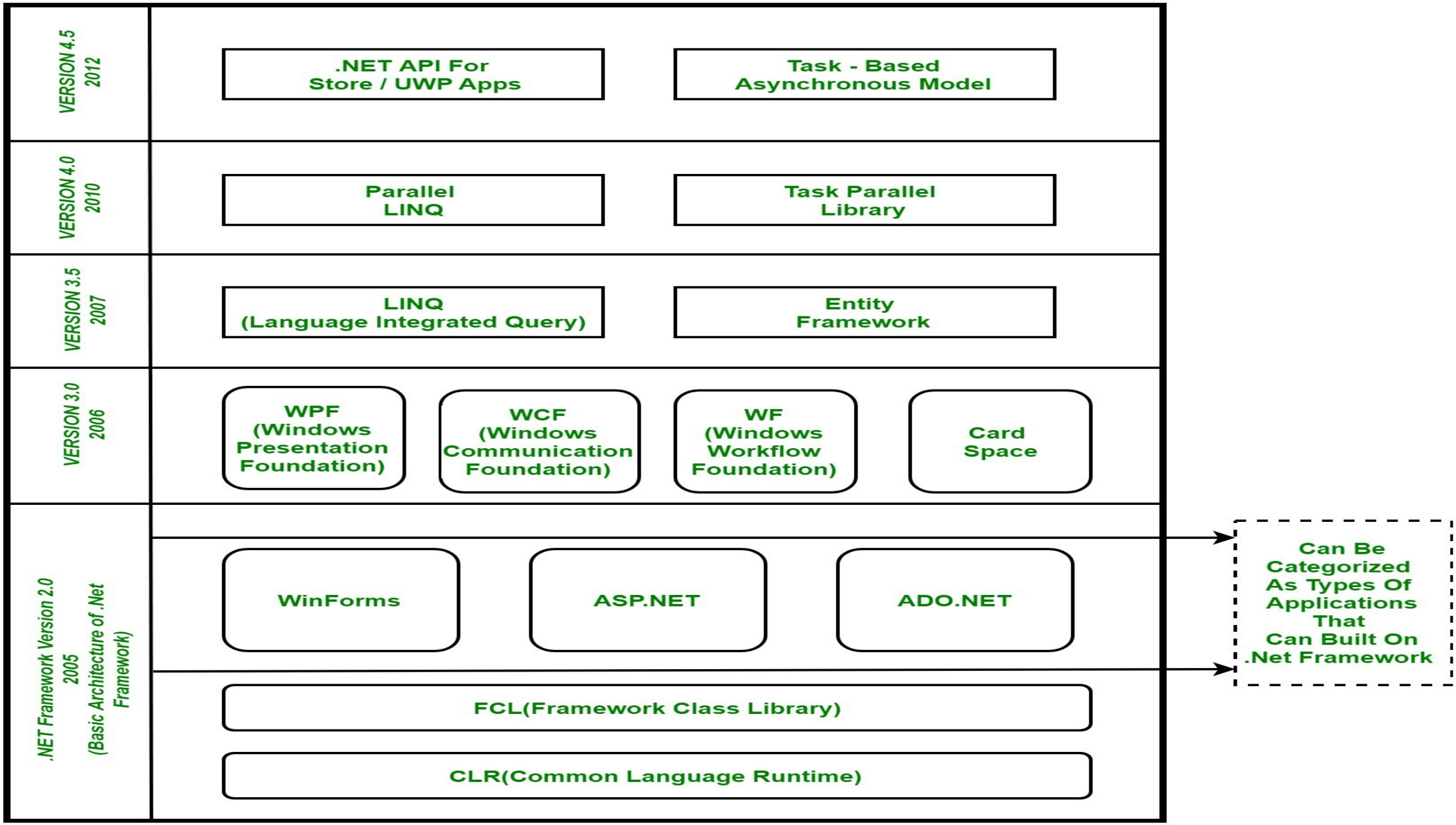
- Provide advanced functionality for complex applications.
  - **Windows Presentation Foundation (WPF):** a graphical subsystem by Microsoft for rendering user interfaces in Windows-based applications.
  - **Windows Communication Foundation (WCF):** a framework for building service-oriented applications.
    - Using WCF, you can send data as asynchronous messages from one service endpoint to another.

Extended Application Modules

- **Windows Workflow Foundation (WF):** a Microsoft technology that provides an API, an in-process workflow engine, and a rehostable designer to implement long-running processes as workflows within .NET applications.
- **Language Integrated Query (LINQ)** is a query language introduced in the .NET 3.5 framework.
  - It is used to make the query for data sources with C# or Visual Basic programming languages.
  - **Note:** AppDomains, WCF, and WF are not supported in .NET Core/.NET 5+.

Extended Application Modules

- **Entity Framework(EF):** an Object-Relational Mapping (ORM)-based open-source framework that is used to work with a database using .NET objects.
  - It eliminates a lot of developers' effort to handle the database.
  - It is Microsoft's recommended technology to deal with the database.
- **Parallel LINQ(PLINQ)** is a parallel implementation of LINQ to objects.
  - It combines the simplicity and readability of LINQ and provides the power of parallel programming.
  - It can improve and provide fast speed to execute the LINQ query by using all available computer capabilities.


VERSION 4.5
2012
.NET API For
Store / UWP Apps
Task - Based
Asynchronous Model
VERSION 4.0
2010
Parallel
LINQ
Task Parallel
Library
VERSION 3.5
2007
LINQ
(Language Integrated Query)
Entity
Framework
VERSION 3.0
2006
WPF
(Windows
Presentation
Foundation)
WCF
(Windows
Communication
Foundation)
WF
(Windows
Workflow
Foundation)
Card
Space
.NET Framework Version 2.0
2005
(Basic Architecture of .Net
Framework)
WinForms
ASP.NET
ADO.NET
FCL(Framework Class Library)
CLR(Common Language Runtime)
Can Be
Categorized
As Types Of
Applications
That
Can Built On
.Net Framework

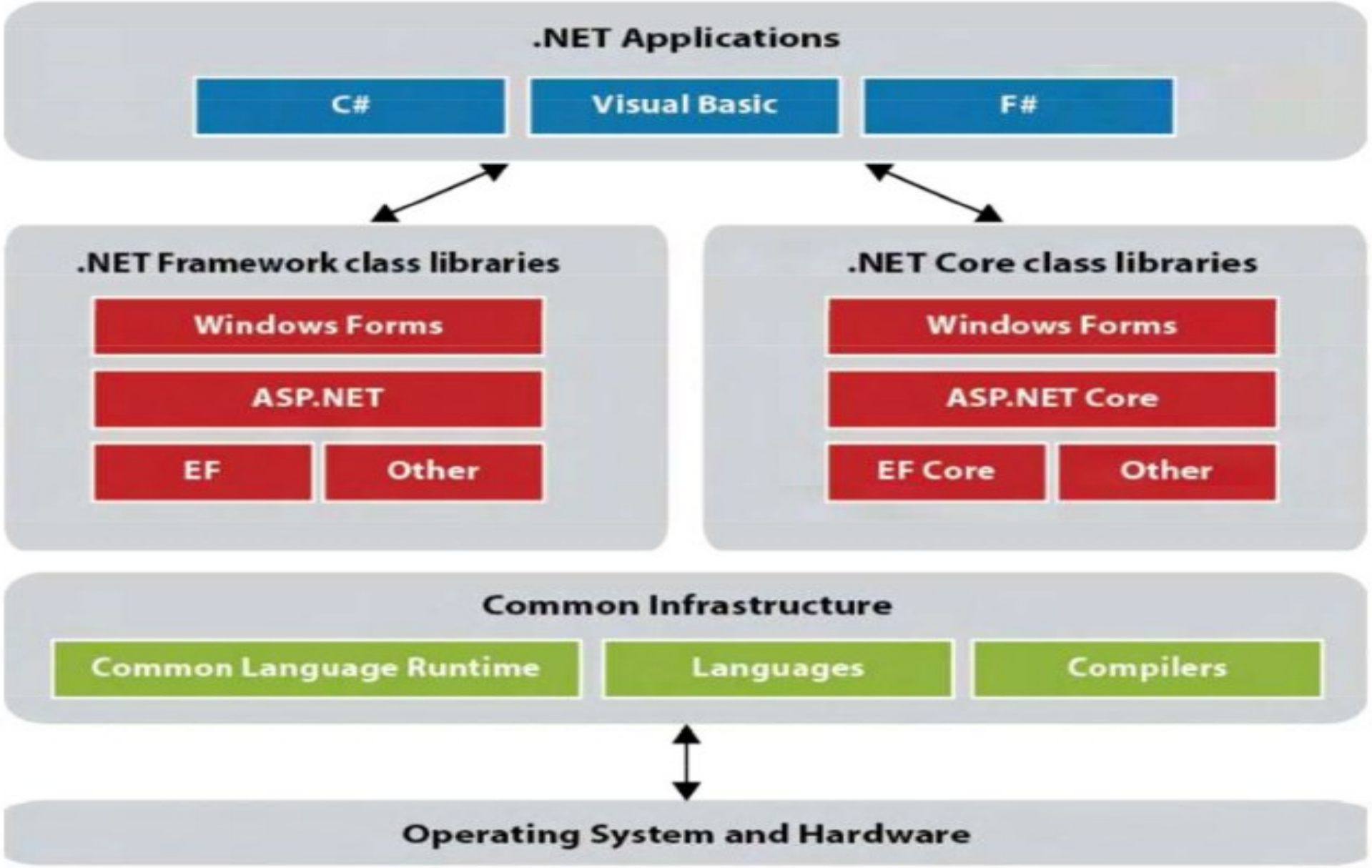
.NET Vs .NET Framework

.NET	.NET Framework
Runs on Linux, macOS, & Windows	Only runs on Windows
Open-source & accepts contributions from community	Source code available but does not accept contributions
All the innovation happens here! Supports more application types and delivers higher performance	Security & reliability bug fixes only
Not shipped with operating system	Included in Windows & updated by Windows updates
Recommended for new development	

.NET Framework / .NET Core

- . **Windows Forms Applications:**
  - Do not access OS/hardware directly.
  - Use .NET services, which interact with the OS and hardware.
- **Class Libraries:**
  - Thousands of pre-written classes available to all .NET languages.
  - Even simple applications can be built with just a few classes.
- **Common Infrastructure:** Includes CLR, languages, and compilers.
- **Common Language Runtime (CLR):**
  - Manages program execution.
  - Handles memory management, code execution, security, and services.
  - Applications are called managed applications.
- **Common Type System (CTS):** Ensures consistent data types across all .NET languages.

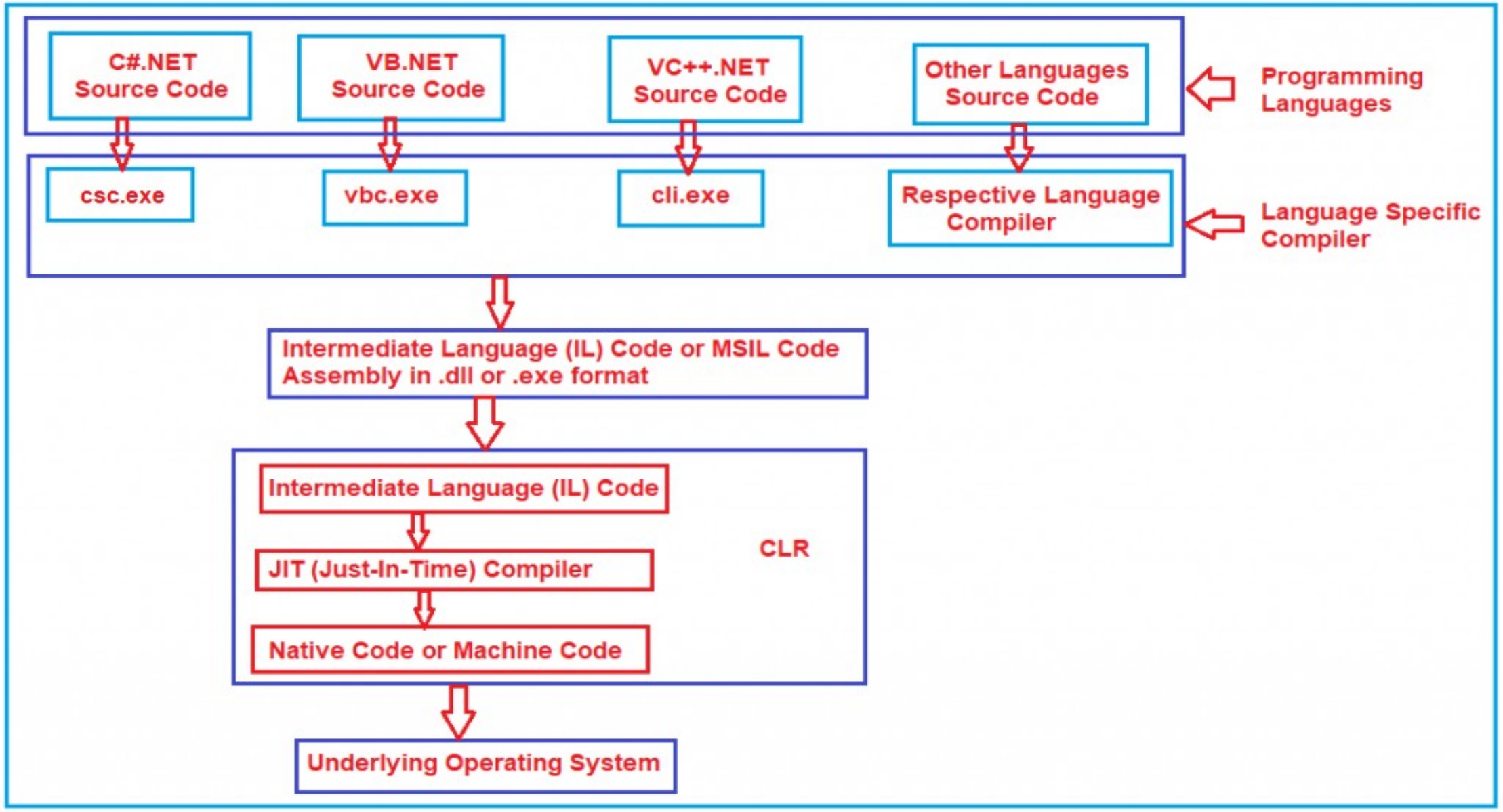
The .NET Framework and .NET Core



.NET Application Compiled and Run

- **First Compilation:**
- Source code is compiled by the language compiler (e.g., C#, VB.NET).
- Generates MSIL (Microsoft Intermediate Language), also known as Managed Code.
- **Second Compilation:**
- CLR uses JIT (Just-In-Time compiler) to convert Microsoft Intermediate Language -> Native Code (specific to OS).
- Native code executes on the operating system.

.NET Application Compiled and Run



Programming Languages

- A language is a set of instructions for communication; human languages vs formal computer languages.
- Computer languages are formal languages that allow programmers to write instructions that are eventually translated into machine-readable code.
- Computers inherently understand only binary (machine code).
  - Without higher-level abstractions (languages, compilers), programming would be complicated.
- Programming languages enable human-readable expressions of logic, which are then converted (via compilers or interpreters) into machine code.

Programming Languages

- **Level of Abstraction**
  - **Low-level languages:** close to machine code &provide direct control over hardware.
    - Machine Language: Binary instructions directly executed by the CPU.
    - Assembly Language: Symbolic representation of machine instructions using mnemonics, e.g., MOV, ADD.
  - **High-level languages:** More human-readable, abstract hardware details support complex data structures and control flow, such as Python, Java, C++, C#, and more.
- **Intended Use:**
  - **General-Purpose Languages:** Designed to solve problems across domains. Examples, Python, Java, C++, C#.
  - **Domain-Specific Languages:** Designed for a specific application domain. Examples, SQL (database), HTML/CSS (web page structure and styling), R (statistical computing).

Programming Languages

- **Programming Paradigm:**
  - **Imperative Programming:** Tells how to do something step by step.
    - Procedural Programming: Code organized into functions/procedures. Examples, C, Pascal.
    - Object-Oriented Programming: Code organized into objects encapsulating data + behavior. Examples, Java, C++, Python, C#.
  - **Declarative Programming:** Focuses on what the program should accomplish.
    - Functional Programming: Based on mathematical functions, it avoids side effects. Examples, Haskell, Scala, Erlang.
    - Logic Programming: Uses facts and rules to infer results. Example, Prolog.
  - **Scripting Languages** (a practical category, though not always a paradigm): Usually interpreted, used for automation, glue code, or extending applications. Examples, JavaScript, Python, Ruby, PHP, Bash.

How Do Computer Programs work?

- A program is a set of instructions given to a computer to perform a specific task.
- **Need for Translators:**
  - Computers understand only binary (0s and 1s); programmers write in high-level languages.
  - To bridge this, we use translators (compilers, interpreters, assemblers).
    - **Compilers**: Translate the entire high-level source code into machine code in one go. Faster execution once compiled, like C, C++, C#, Java
    - **Interpreters**: Translate and execute code line by line at runtime. Easier to test/debug per line; errors can stop execution at the error line. Slower overall. E.g., Python, Ruby, JavaScript.
    - **Assemblers**: Convert assembly (low-level human-readable code) to machine code. E.g., Microsoft Assembler, Netwide Assembler, GNU Assembler.

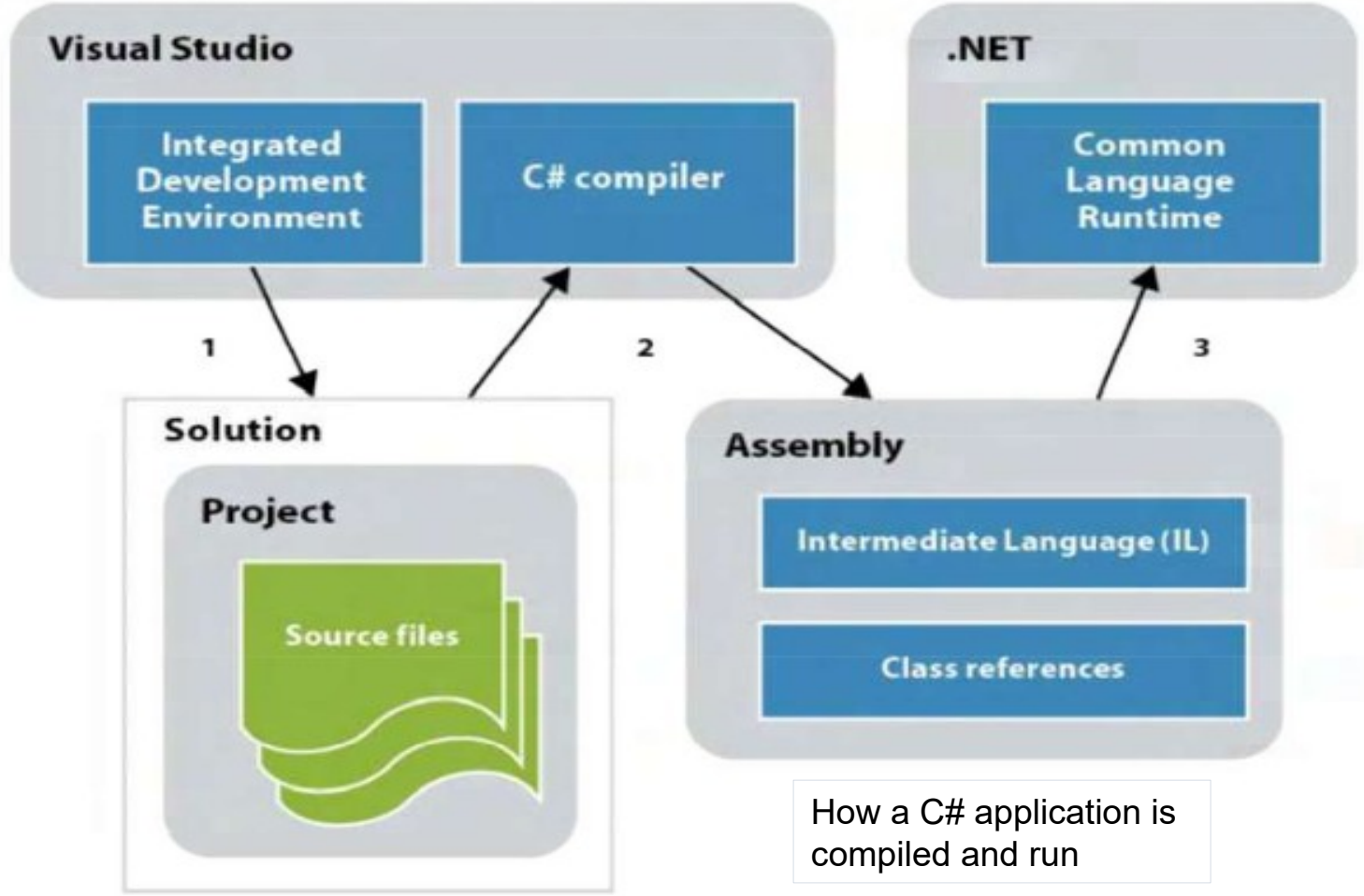
How Do Computer Programs work?

- **Operating System Role:**
  - OS acts as the master program auto-loaded in memory on startup; it handles memory management, process management, input/output, etc.
  - Required for running any other programs.
- **Loader, Locator, Linker:**
  - Loader: loads machine code into system memory.
  - Locator: assigns memory addresses.
  - Linker: combines modules/subprograms into a single executable or library.

C# (C-Sharp)

- A modern, general-purpose programming language developed by Microsoft, designed for the Common Language Infrastructure (CLI), enabling cross-platform execution via the .NET runtime.
- C# is pronounced as *C-Sharp*.

- Projects are created in Visual Studio with C# source and resource files.
- The C# compiler translates source code into Intermediate Language (IL).
- Compiled code is stored in an assembly (.exe or .dll).
- The CLR executes assemblies, managing code, memory, and security.
- A solution is a container holding one or more projects.



How a C# application is compiled and run

C# (C-Sharp)

C# Paradigms and Modern Features:

- **Object-Oriented + Functional:** Incorporates features like delegates, lambdas, and LINQ for a hybrid programming experience.
- **Language Integrated Query(LINQ):** Enables powerful, type-safe data querying over collections, databases, XML, and more.
- **Generics and Partial Classes:** Type-safe reusable code via generics; modular class definitions using partial classes.
- **Dynamic & Async Programming:** Enables runtime-type resolution for interop scenarios and simplifies asynchronous programming for responsive UIs and scalable servers.
- **Compiler Platform-Roslyn:** Open-source, self-hosted compiler infrastructure offering powerful code analysis and tooling APIs.
- The latest version of C# 13 was released in November 2024.

C# (C-Sharp)

- **Strengths of C#:**
  - **Object-Oriented and Component-Oriented:** Supports classes, inheritance, polymorphism, encapsulation, and modular design.
  - **Type-Safe and Robust:** Prevents unsafe memory access; includes garbage collection and exception handling.
  - **Simple and Familiar Syntax:** Easy to learn for C, C++, Java, or JavaScript developers.
  - **Cross-Platform:** Runs on Windows, Linux, & macOS via .NET Core/.NET 5+.
  - **Performance:** Compiled to Intermediate Language (IL) and executed by the Just-In-Time (JIT) compiler for efficient runtime performance
  - **Rich Ecosystem:** Libraries, frameworks (ASP.NET, MAUI, Unity), and strong community support.

C# (C-Sharp)

- We can develop different types of secure and robust applications by using the C# programming language:
  - Desktop and Windows applications using WPF, WinForms, MAUI.
  - Web applications and APIs using ASP.NET Core, Blazor
  - Distributed systems and microservices
  - Web services and backend services
  - Database-centric apps using Entity Framework or direct ADO.NET etc.

Event-Driven Programming with C#: IDE and Editor

- **Integrated Development Environment(IDE):** A complete software suite that provides all tools needed for development in one place.
  - The IDE has a Code Editor with IntelliSense, syntax highlighting, auto-complete, compiler/interpreter, debugger, project management Tools, UI designers, and testing tools. For example, Visual Studio, Eclipse, JetBrains Rider,...
  - **Visual Studio** is a **64-bit application created by Microsoft**, used for building, debugging, and deploying applications across platforms such as Windows, web, mobile, cloud, and more, supporting languages like C#, C++, Python, and JavaScript.
- **Editor:** A lightweight tool mainly for writing and editing source code with basic features like syntax highlighting and extensions/plugins for added functionality.
  - Relies on external tools such as a compiler and debugger for full development, for example, Visual Studio Code, ...

Visual Studio IDE

Visual Studio 2022 Hardware Requirements

CPU / Processor

Minimum: 64-bit ARM64 Intel64 or x64 | Recommended: Quad-core or better

RAM

Minimum: 4 GB | Recommended: 16 GB or more

Disk / Storage

Minimum: 850 MB | Recommended: 20-50 GB free, SSD preferred, or HDD

Display / Graphics

Minimum: WXGA (1366×768) | Recommended: Full HD (1920×1080) or higher

Operating System / Platform

Minimum: 64-bit Windows 10 or newer | Recommended: Same with updates

Visual Studio 2022 Software Requirements

.NET Framework

Requires .NET Framework 4.5.2 or later

Workloads & SDKs

Involves selecting necessary workloads and SDKs

Windows SDK

Includes Windows SDK for app development

Updates & Patches

Emphasizes the need for up-to-date system components

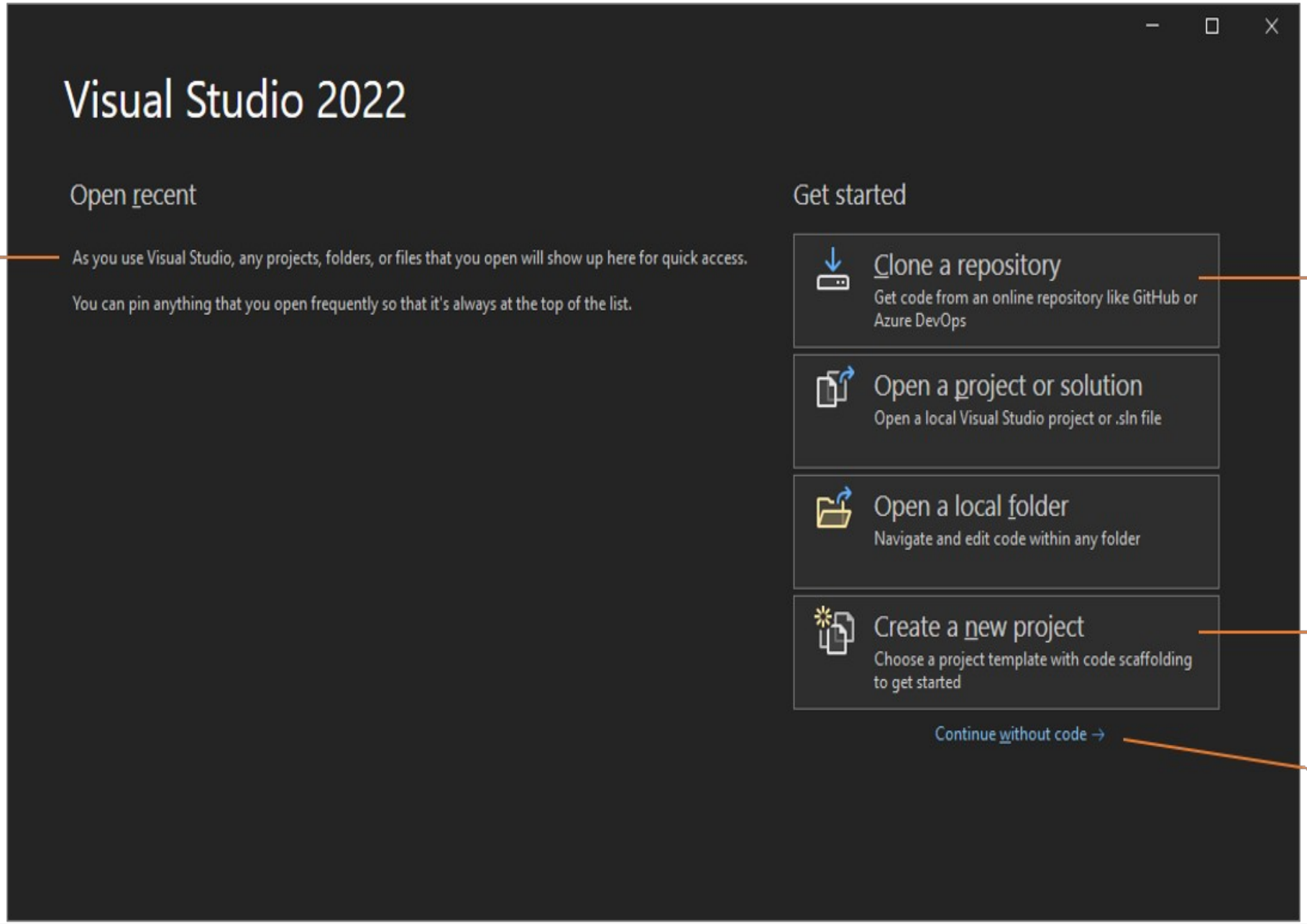
Other Dependencies

Lists additional tools and considerations

Visual Studio IDE

- **Download and Install Visual Studio 2022 IDE:**
  - **Link**: https://visualstudio.microsoft.com/downloads/
- **Select and Install Workloads during Installation:**
  - **.NET Desktop Development:** workload for Windows Forms, WPF, Console Applications,...
  - **ASP.NET and Web Development:** workload for web applications and services.
  - **Data Storage and Processing:** workload, for use with Microsoft SQL Server integration
- **Installation Procedure Link**: https://learn.microsoft.com/en-us/visualstudio/ide/quickstart-ide-orientation?view=vs-2022

Visual Studio IDE: Start Window



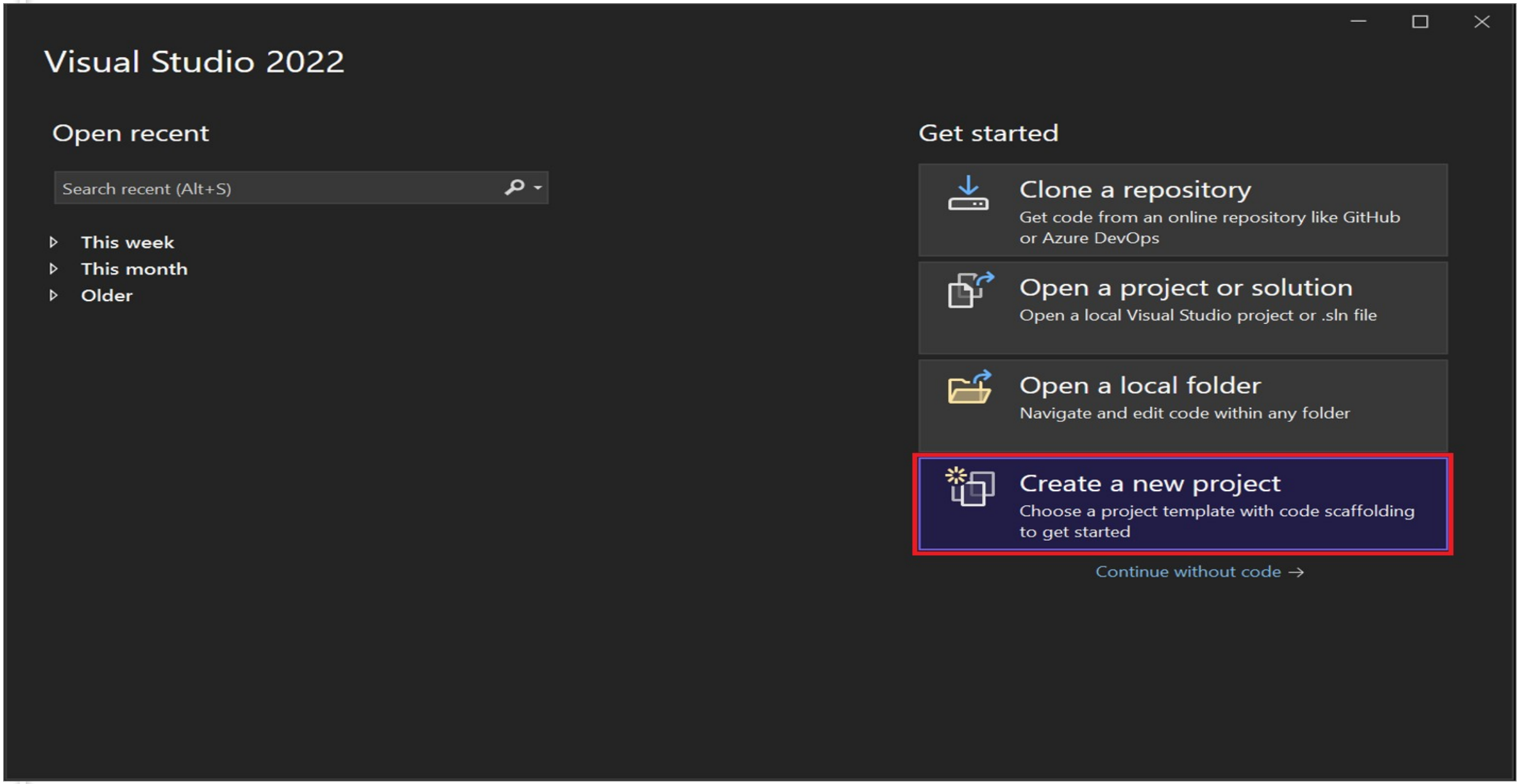
Open a project that you used recently

Collaborate on existing code

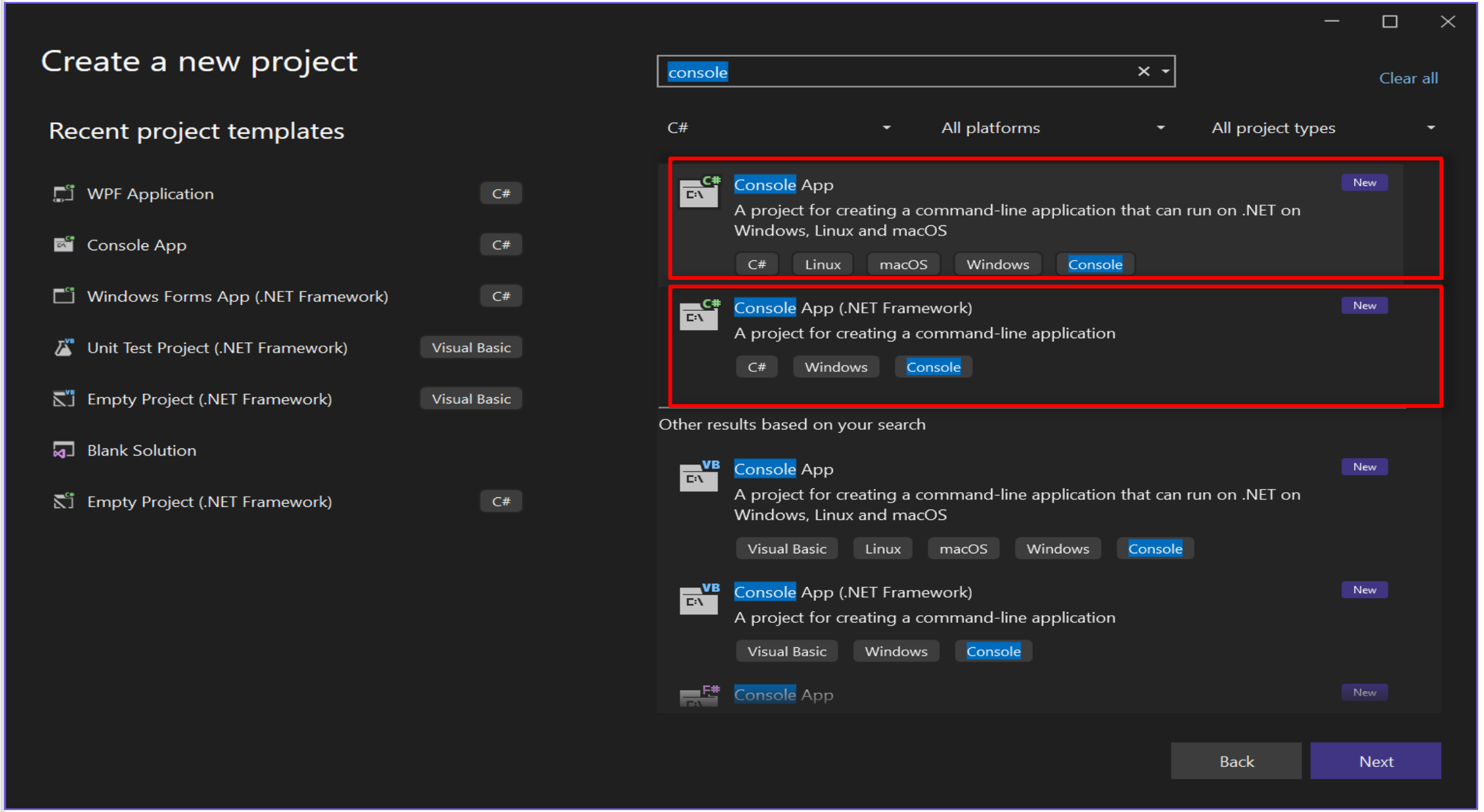
Start fresh with a brand new project

Open Visual Studio without any code

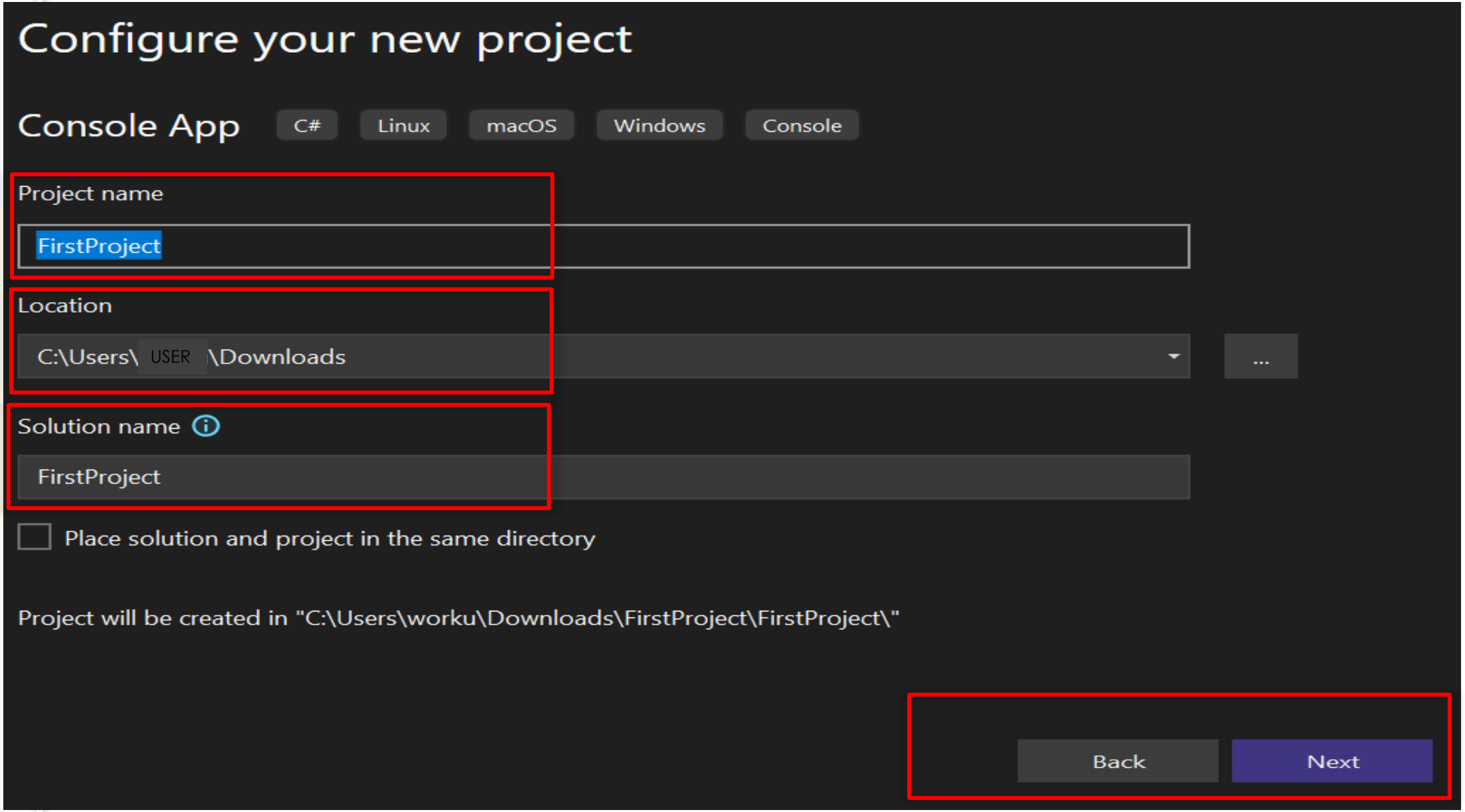
Visual Studio IDE: Create New Project



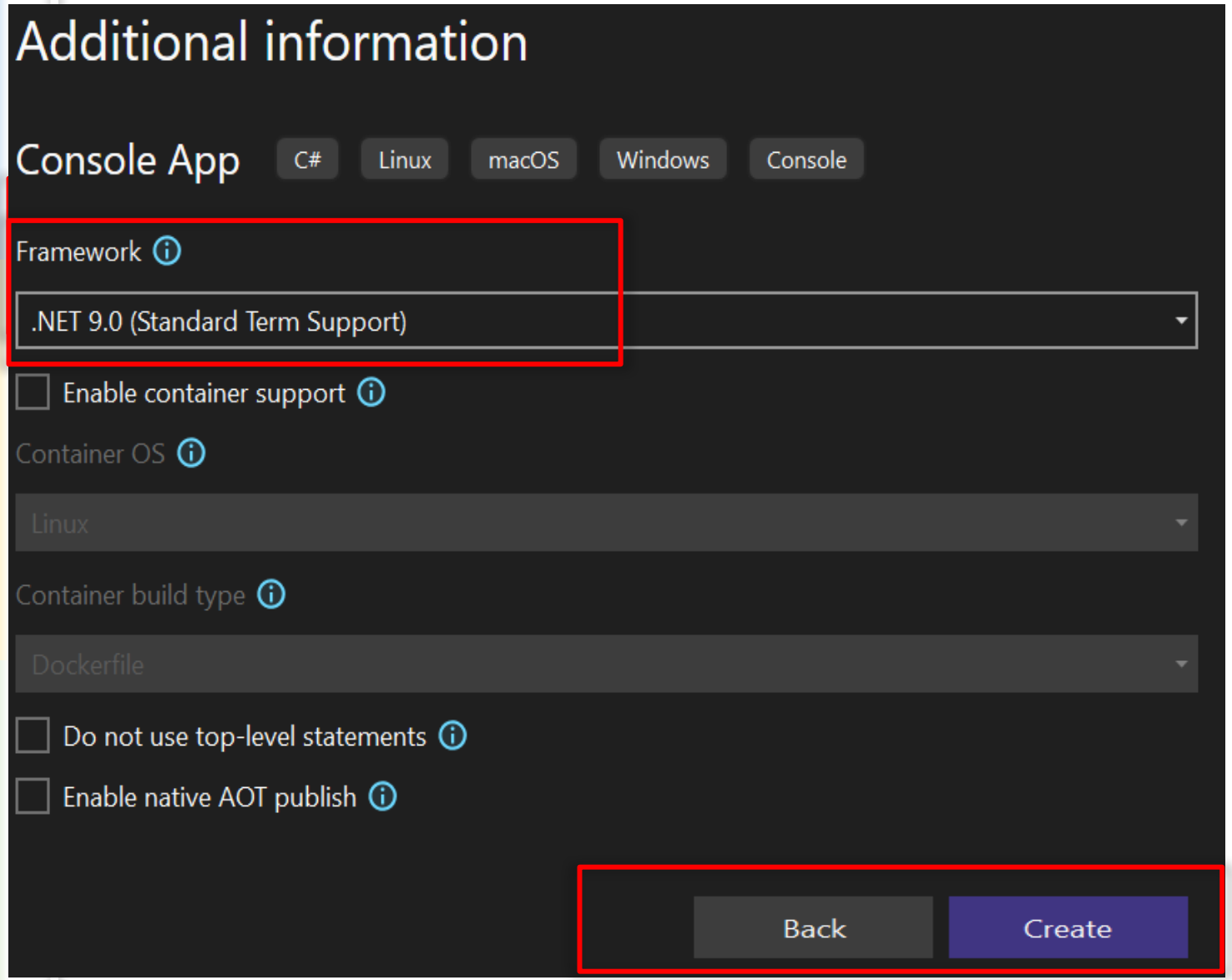
Visual Studio IDE: Create New Project Windows

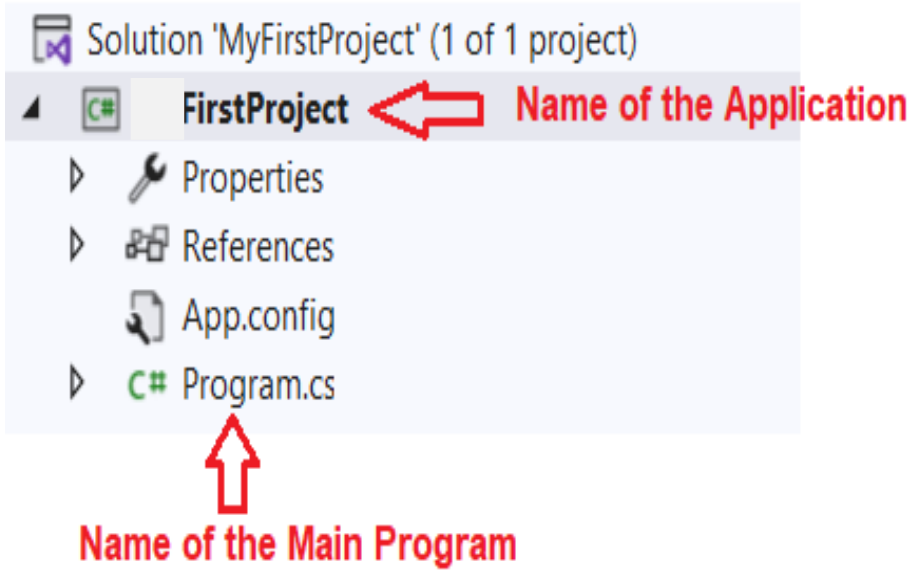


Visual Studio IDE: Configure Your New Project



Visual Studio IDE: Additional Information





A project called **FirstProject** will be created in Visual Studio.

Thank You!

?